

RAG-Minimal 学习手册

面向 **只有 Python 基础** 的学习者：从零看懂并跑通一个手写 RAG（检索增强生成）。配套单文件 `rag_minimal.py`（164 行、零第三方依赖）——一份代码 + 一份手册，没有别的。

0. 开始之前：两份基础自测

0.1 Python 基础

本手册默认你会这些（都是 Python 入门就会的）：

- 变量、函数（`def` + `return`）、`if / for / while`
- 列表、字典、元组（`[1,2]` `{"a":1}` `(1,2)`）
- 字符串拼接、`print()`

看不懂的 Python 知识点（类型注解 `-> list[str]`、生成器表达式、`zip`）手册会**边用边讲**，不是前置要求。

基础不牢，先补这三个免费优质资源（全部免费）：

资源	类型	地址
菜鸟教程 · Python3	图文教程，查语法快	https://www.runoob.com/python3/python3-tutorial.html
廖雪峰 · Python 教程	图文教程，体系完整	https://liaoxuefeng.com/books/python/
林粒粒呀 · Python 入门课	B 站视频，讲得透	https://www.bilibili.com/video/BV1Jgf6YvE8e/

建议顺序：廖雪峰通读前几章（列表 / 字典 / 函数 / 循环）→ 菜鸟教程当字典查 → 林粒粒呀视频配着看。**别贪多，能写出“遍历一个列表、拼接字符串”就够进本手册了。**

0.2 Markdown 基础（切块一节要用）

本工程的文档用 Markdown 写的，切块就是靠它的**标题语法**来识别边界。如果你还没接触过 Markdown，先花十分钟过一遍（会写 `# 一级标题`、`## 二级标题`、`- 列表项` 就够）：

资源	说明	地址
Markdown 官方教程（中文）	最标准，入门看这个	https://markdown.com.cn/basic-syntax/
菜鸟教程 · Markdown	查语法快	https://www.runoob.com/markdown/md-tutorial.html
Markdown 中文文档	更全的参考	https://www.markdown.cn/docs/intro/

为什么要懂 Markdown？因为

RAG 的 "切块" 吃的是文档结构

—— 哪里是标题、哪里是正文、哪里是代码块，全靠 Markdown 语法来认。不懂语法就看不懂 "为什么这样切"。

0.3 模型服务：先起两个本地模型（照抄即可）

RAG 链路要两个模型：**嵌入模型**（把文字变向量）+ **对话模型**（把检索结果变成人话）。两个都是 llama.cpp 起的本地服务，一个占 8081 端口、一个占 8080 端口：

```
# 终端 1：嵌入服务（8081 端口）
llama-server -m D:\模型路径\嵌入模型.gguf --embedding --pooling last -ngl 999 -c 8192 --port 8081

# 终端 2：对话服务（8080 端口）
llama-server -m D:\模型路径\对话模型.gguf -ngl 999 -c 8192 --port 8080
```

参数含义：

参数	作用
<code>--embedding --pooling last</code>	嵌入模式 + last 池化。Qwen 系嵌入模型必须带，否则向量不对
<code>-ngl 999</code>	把模型层全放到显卡（有 NVIDIA 独显时加；纯 CPU 电脑删掉这个参数）

参数	作用
<code>-c 8192</code>	上下文窗口大小

模型推荐（原理版不需要大模型）：

- 嵌入：Qwen3-Embedding-0.6B（约 0.6GB）—— CPU 也能飞快
- 对话：Qwen3-1.7B（约 1.1GB）或 Qwen3-4B（约 2.4GB）

原理一样，模型越小越流畅——RAG 的骨架跟模型大小无关，别上来就用 8B / 27B 把自己卡死。

卡成 PPT / 内存高？看这条

症状	原因	解决
提问后卡几十秒到几分钟	Qwen3 系默认开"思考"：先输出一大坨英文推理才回答	新版 <code>rag_minimal.py</code> 已带 <code>chat_template_kwargs: {"enable_thinking": False}</code> + <code>max_tokens: 512</code> ，升级代码即可
内存 / 显存高	没带 <code>-ngl</code> 时模型全在 CPU 跑（慢且吃内存）；或模型太大（8B / 27B）	有独显加 <code>-ngl 999</code> ；没独显就用 0.6B / 1.7B 小模型

这正好回答"原理版是不是拼硬件"：**不是**。原理版全程跑最小模型（0.6B + 1.7B），任何能装 Windows 的电脑都流畅；大模型是生产版（RAG_HandProj）的事。

1. 先跑起来（照做就行）

前置：一台电脑 + Python 3.10+，两个模型服务已启动（怎么启动见 0.3 节）。

```
# 1. 建库：把 Markdown 文档切块 + 向量化 + 存入 SQLite
python rag_minimal.py --build 你的笔记.md

# 2. 提问：检索 + 生成，答案下方带参考源
python rag_minimal.py --ask "这份文档讲了什么？"

# 3. 清空知识库（重来一次用）
python rag_minimal.py --reset
```

跑通了再往下读。读代码时对照你刚跑的结果，每一节都能找到你看到的输出。

2. 程序长什么样：从入口读起

Python 程序从 `if __name__ == "__main__":` 开始执行：

```
if __name__ == "__main__":  
    main()
```

Python 新手必问的写法： `__name__` 是内置变量。直接运行本文件时它等于 `"__main__"`，会执行 `main()`；被别的文件 `import` 进来时它等于模块名，**不会执行**——这样别人 `import` 你的代码不会误触发运行。

`main()` 用 `argparse` 解析命令行参数，三条路：

```
if args.reset:      # --reset: 清库  
    ...  
elif args.build:    # --build 文档.md: 建库  
    store(chunk(Path(args.build).read_text(encoding="utf-8")))  
elif args.ask:      # --ask "问题": 提问  
    generate(args.ask)
```

注意 `--build` 那一行是个**函数套函数**：`chunk(...)` 先把文本切成块，`store(...)` 再把块存进库——**先切块，再入库**，这就是 RAG 建库的两步。往下每一节拆一个函数。

3. 第一步：切块 (chunk)

为什么切

模型的“眼睛”（上下文窗口）一次只能看有限文字，整本书塞不进去。所以先把文档切成小块，提问时只挑相关的几块喂给它。**切块是检索质量的第一道闸门**——块切得好不好，直接决定后面能不能检索到。

切法分两类：固定窗口 vs 非固定

固定窗口切：不管内容，每 500 字一刀。实现最简单，但刀可能砍在句子中间，把一句完整的话劈成两半——**语义断裂**。

非固定窗口切：找“语义边界”下刀——标题、段落、空行、代码块都是边界。语义完整的单元一块，**切完语义不断**。按标题切就是非固定切法的一种形式。

这里的关键不是“按标题切”这个具体招式，而是**因地制宜**：你的文档是什么结构，就选什么边界。比如：

文档长什么样	合适的非固定切法
有标题层级（Markdown/Word 转的）	按标题切
没有标题，但有空行分段	按空行 / 段落切
纯代码	按代码块 / 函数切
全是正文没有结构	才退回固定窗口（加重叠保语义）

目的只有一个：向量化之前，语义信息和上下文不断裂。断了的块，向量也是残缺的，检索就会漏。

本工程为什么用标题切

本工程文档都经 anydoc 转成了带结构的 Markdown（标题、正文、代码块分得清清楚楚），所以**标题切在这里最适用**——这是 "因地制宜" 的选择，不是唯一答案。

代码怎么切

先判标题：

```
def _is_head(s: str) -> bool:
    if s.startswith("【") and s.endswith("】"): # 手册原生标题：【xxx】
        return True
    return s.startswith("#") and bool(s.lstrip("#").strip()) # Markdown 标题：# xxx
```

`startswith` / `endswith` 是字符串方法，判断 "以什么开头 / 结尾"。第二行三个连环操作拆开看：

- `s.startswith("#")`：这行以 `#` 开头吗？
- `s.lstrip("#")`：把开头所有 `#` 删掉（`lstrip` = left strip，只删左边）
- `.strip()`：再把首尾空格删掉
- `bool(...)`：删完还有内容吗？有就是 `True`

所以 `# 标题` 是标题，而孤零零的 `#`（没有文字）不是——**标题必须有内容**。

再按标题分块：

```
for raw in text.splitlines(): # splitlines: 按行拆成列表，一行一个元素
    s = raw.strip()           # strip: 去掉首尾空白（空格/换行）
    if s.startswith("`"):    # 代码块开关
```

```

    in_code = not in_code
    if not in_code and _is_head(s): # 在代码块外，且是标题
        if current:
            blocks.append("\n".join(current)) # 把攒好的上一节收尾
            current = [s] # 新开一节，标题是这一节的第一行
        elif s:
            current.append(s) # 普通行：追加到当前节

```

核心是 `current` 这个 "攒行桶"：读到标题就把桶里攒的旧内容倒进 `blocks`，再开新桶。循环结束再倒一次（文档最后一段没有 "下一个标题" 触发收尾）。

两个防坑设计：

```

if not head_lines: # 完全没有标题 → 固定窗口切兜底
    return [text[i : i + WINDOW] for i in range(0, len(text), WINDOW - 50)]

```

- **兜底**：文档完全没有标题时，退回固定窗口切。步长 450（500 字一刀，重叠 50）—— 重叠是给 "语义断裂" 补的保险：一句话前半截在上一块、后半截在下一块时，重叠让换块也能找到完整语义。**两个拼图故意重叠一条边，拼缝切不到字上**
- **代码块保护**：````` 内不判标题。否则 Markdown 代码块里的注释 `# 这是注释` 会被误判成标题，把代码块切成碎片

4. 第二步：向量化 (embed)

电脑不懂 "意思"，但懂数字。嵌入模型把一段文字映射成一串数字（向量），规则：**意思相近的文字，向量方向也相近**。

```

def embed(text: str) -> list[float]:
    return _post(EMBED_URL, {"model": "x", "input": text})["data"][0]["embedding"]

```

`_post` 做的事：把字典转成 JSON、POST 给本地模型服务、读回响应。这是 **HTTP 请求**—— 你的程序在跟另一个程序 (llama-server) 说话：

```

def _post(url: str, payload: dict) -> dict:
    req = urllib.request.Request( # 造一个请求
        url, data=json.dumps(payload).encode(),
        headers={"Content-Type": "application/json"},
    )

```

```
with urllib.request.urlopen(req, timeout=60) as resp: # 发出去, 等响应
    return json.loads(resp.read()) # 读回字节 → 解析成字典
```

新手最懵的是**编码**: `json.dumps(payload).encode()` 把字典变成 JSON 字符串再变成字节 (网络只传字节), `json.loads(resp.read())` 把响应字节还原成字典。**请求是 "字典→字节", 响应是 "字节→字典", 对称的。**

关键认知: **向量是一串数字, 数字本身没意义, 方向才表达语义。**"猫" 和 "猫咪" 的向量方向很近, "猫" 和 "汽车" 很远。所以比较两段文字像不像, 比的是方向 —— 第 6 节。

5. 第三步: 存储 (store)

为什么用 SQLite 存向量

先亮明立场: **本工程重在原理理解, 所以用 SQL 数据库 (SQLite) 存向量。**理由:

- **开箱即用**: SQLite 是单文件数据库, Python 自带 `sqlite3`, 不用装驱动、不用起服务
- **足够简单**: 理解 "存进去、查出来" 这个原理, 完全够用

SQLite 不是 "不能存向量", 而是**没给检索建索引**——检索时要全表扫描, 把向量一条条取出来算余弦。数据量小时这缺点完全不是事, 数据量大了才变成瓶颈。

"小数据" 是多大? SQLite 官方定位是**嵌入式、单机数据库**: 单表几百万行、单机低并发读写毫无压力, 个人知识库 (几千到几万块) 对它来说是小菜一碟。但向量检索是另一回事——**全表扫描算相似度**: 几千块秒回, 几十万条就开始卡。所以量级到 "生产级" (百万级数据、毫秒响应、高并发) 才换向量数据库。原理上两者一样, 区别在 "谁来做检索" (见下)。

存什么、怎么存

存三样: 块的正文 (给模型看)、块的向量 (给检索用)、块的指纹 (给去重用)。

```
con = sqlite3.connect(DB) # 连数据库 (没有就建文件)
con.execute("CREATE TABLE IF NOT EXISTS chunks(...)")
for b in blocks:
    h = hashlib.sha1(b.encode()).hexdigest() # ① 指纹: 内容 → 固定长度哈希
    vec = embed(b) # ② 向量: 每个块向量化
    blob = json.dumps(vec).encode() # ③ 转字节: 向量 → BLOB
    con.execute("INSERT OR IGNORE INTO chunks(hash, content, vector) VALUES(?,?,?)", (h,
```

```
b, blob))
con.commit() # 提交, 写入磁盘
con.close()
```

三个关键点, 按顺序理解:

- **BLOB 存向量** (一开始就要明白): 向量是一串浮点数 (`[0.12, -0.34, ...]`), 不能直接塞进 SQLite 的文本列。 `json.dumps(vec).encode()` 把它变成 JSON 字符串再编码成字节 (BLOB 类型), 省空间、查询快。取出来时反向操作: `json.loads(bytes)` 还原成列表 —— 第 6 节你会看到
- **hash 幂等去重**: `sha1` 把内容算成固定长度指纹 (内容相同指纹必相同)。 `INSERT OR IGNORE` = "插不进去就忽略" —— 同一文档重复上传, hash 撞了直接跳过, **不会攒出重复块**
- **?? 占位符**: SQL 里的 `?` 是 "这里放一个值", 真正的值由第二个参数元组提供。这是防注入的标准写法, **永远不要把变量直接拼进 SQL 字符串** (拼接会出 SQL 注入漏洞, 新手必踩)

真实世界用什么: 向量数据库

生产里的检索不在你的 Python 代码里做, 而在向量数据库内部完成:

原理版 (本工程):

你的代码遍历库里所有向量 → 逐个算余弦 → 自己排序 → 取前 3

真实世界 (pgvector / Milvus / Chroma):

你的代码把"问题向量"交给数据库 → 数据库内部算好相似度 → 直接返回一个按相似度从高到低排好的列表

差别在哪? 原理版把 "遍历 + 排序" 摊在你的 Python 循环里; 向量数据库把这步下沉到数据库内部 (建了向量索引, 不用全表扫描, 几十万条也能毫秒级返回)。但原理是同一个: **拿问题向量和每条数据算相似度, 按相似度从高到低排, 取前 k**。看懂原理版的遍历, 就懂了向量数据库在干嘛。

6. 第四步: 检索 (search)

余弦相似度的数学

两个向量的 "像不像" 用**方向夹角**衡量: 夹角越小越像。余弦相似度 = 点积 ÷ (模长 × 模长), 范围 -1 到 1。

```
def _cosine(v1: list[float], v2: list[float]) -> float:
    dot = sum(x * y for x, y in zip(v1, v2)) # 点积: 对应位相乘再求和
```

```
n1 = sum(x * x for x in v1) ** 0.5          # 模长：每个数平方求和再开方
return dot / (n1 * n2) if n1 and n2 else 0.0
```

`zip(v1, v2)` 把两个列表**按位配对**： `[(v1[0],v2[0]), (v1[1],v2[1]), ...]`。 `sum(x*y for x,y in ...)` 是生成器表达式——`for` 循环的 "压缩版"，逐个算 `x*y` 再求和。`** 0.5` 就是开平方（`x**0.5 == √x`）。

为什么用余弦不用距离（欧氏距离）？ 距离受 "向量长度" 影响：长文本的向量天然更长，距离就大，但语义未必不相关。余弦**只比方向，不管长短**——公平。就像比较两个人的观点：问 "方向一不一致"（余弦），而不是 "离得多远"（距离）。

原理版检索 = 遍历 + 排序 + 取前 k

```
scored = sorted(
    ((content, _cosine(qv, json.loads(v))) for content, v in rows), # 遍历每条数据算相似度
    key=lambda p: p[1], reverse=True,      # 按相似度从高到低排序
)
return scored[:k]                          # 取前 k 个 (k=3)
```

- `json.loads(v)`：把库里存的向量字节还原成列表（第 5 节存的，这里取回来）
- 遍历库里每一条：`(正文, 相似度)` 组成元组
- `sorted(..., key=lambda p: p[1], reverse=True)`：按相似度（元组第 1 位）从高到低排
- `scored[:k]`：切片取前 3——**最像的 3 块，就是给大模型的资料**

和向量数据库的差别：这里遍历 + 排序摊在 Python 列表里，数据库版下沉到库内——**同一套原理，只是 "谁来做" 不同**（第 5 节讲过）。

7. 第五步：生成 (generate)

检索到 top-3 块后，把它们拼进提问，明确指令 "根据以下资料回答，不能编造"：

```
prompt = "\n".join(f"资料{i+1}: {c}" for i, (c, _) in enumerate(hits))
prompt += f"\n问题: {query}\n请仅依据以上资料回答，不要编造资料外的内容。"
reply = _post(CHAT_URL, {
    "model": "x",
    "messages": [{"role": "user", "content": prompt}],
```

```
"max_tokens": 512, # 限总输出长度，防无限生成
"chat_template_kwargs": {"enable_thinking": False}, # Qwen3 系默认开思考，关掉才答得快
})
print("答案: ", reply["choices"][0]["message"]["content"])
```

`f"资料{i+1}: {c}"` 是 **f-string**: `{变量}` 会把变量值塞进字符串。 `enumerate(hits)` 给列表元素带编号 (`(0, 第1块)`, `(1, 第2块)`...) , 所以 `i+1` 从 1 开始编号资料。

多出来的两个参数是性能保险: Qwen3 系默认会先输出一大坨英文"思考"再回答(慢且吃内存), `enable_thinking: False` 直接关掉; `max_tokens: 512` 限制总输出长度, 模型就算想无限生成也到不了——这两行是"原理版也能跑得动"的关键(详见 0.3 节)。

这一步把"找到"变成"说人话": 模型基于你给的资料作答, 资料里没有的它会诚实说不知道——**回答可追溯, 错了能查到是哪块资料的问题**(所以 `generate` 会打印参考源和相似度)。

8. 六个动手实验 (改参数看行为变化)

实验 1: 切块重叠——把 `WINDOW` 改成 500、步长 500 (`range(0, len(text), 500)`, 无重叠), 问"一句话被切成两半"的内容, 看检索是否变差。**体会: 重叠是给固定窗口切补语义的。**

实验 2: 代码块保护——在文档里写一个含 `# 注释` 的代码块, 看切块结果: 保护开启时注释在代码块内, 注释前没有新块。**体会: 为什么 Markdown 语法是切块的依据。**

实验 3: top-k——把 `TOP_K` 从 3 改成 5, 看参考源变多后答案是否更全(也会更杂)。

实验 4: 余弦 vs 距离——把 `_cosine` 换成欧氏距离 `sum((x-y)**2 for x,y in zip(v1,v2)) ** 0.5` (注意排序方向反转), 看长文本块的排名怎么被"长度"干扰。

实验 5: 幂等——同一文档 `--build` 两次, 第二次输出"去重后库内共 N 块", N 不变。

实验 6: 空库兜底——`--reset` 后直接 `--ask`, 看提示"知识库是空的"。

每个实验问自己: **改一个参数, 行为变了没有? 变了就说明你理解了它控制什么。**

9. 更进一步: 从原理到页面

RAG-Minimal 是**原理版**——命令行一问一答, 把 RAG 的骨架摊在 164 行里给你看。它离"能用"还差一层, 但**往哪儿走、走多远, 由你定。**

这里有一份路线图(不是标准答案, 是选择题):

方向	做什么	收获
简易前后端	加个 FastAPI + 网页：传文档、提问、看参考源	原理理解变成页面上轻松使用 —— 这是向 "生产版" 进一步的第一步
换向量数据库	把遍历余弦换成 pgvector / Chroma，数据量上去后体验毫秒级检索	理解 "下沉到库内" 到底快在哪
工程化	uv 管理依赖、ruff/mypy/pytest 三件套、git 分支规范	从 "能跑" 到 "能交付"
打包分发	把程序 + 模型打成压缩包 / 安装包 / 单文件便携包，给其他 Windows 用户免环境双击即用	让不懂代码的人也能用上你的东西
部署上线	Docker 打包，或 uv 在云服务器上部署成在线服务	你的作品从本地走向公网

拓展思考题：你能把这 164 行 + 两个模型，变成 "别人下载后双击就能用" 的东西吗？—— 提示方向：压缩包（带启动脚本）、安装包（Inno Setup / NSIS）、单文件便携包、Docker 镜像、云服务器部署。

想自己查资料解决，就动手；卡住了想找个人带，可以来闲鱼找作者咨询 —— **陪跑带你完成打包 / 上线，也是这个项目价值的一部分。**

先懂骨架，再看血肉。 能逐行解释这 164 行，你就拿到了 RAG 的通行证。

License

AGPLv3 —— 允许商用，但你必须告知使用者源码可免费获取；修改或再分发（含部署为网络服务）后，修改版同样必须以 AGPLv3 开源。模型文件与文档内容版权归各自来源。

全文见 [LICENSE](#)。